MeKi: Memory-based Expert Knowledge Injection for Efficient LLM Scaling

Ning Ding[1†], Fangcheng Liu[1†], Kyungrae Kim[2], Linji Hao[1], Kyeng-Hun Lee[2], Hyeonmok Ko[2], and Yehui Tang[1] ✉
[1] Samsung Research, Beijing, China [2] Samsung Research, South Korea
{ning1.ding, yehui.tang}@samsung.com
† Equal Contribution ✉ Corresponding Author

Abstract

Scaling Large Language Models (LLMs) typically relies on increasing the number of parameters or test-time computations to boost performance. However, these strategies are impractical for edge device deployment due to limited RAM and NPU resources. Despite hardware constraints, deploying performant LLM on edge devices such as smartphone remains crucial for user experience. To address this, we propose **MeKi** (**M**emory-based **E**xpert **K**nowledge **I**njection), a novel system that scales LLM capacity via storage space rather than FLOPs. MeKi equips each Transformer layer with token-level memory experts that injects pre-stored semantic knowledge into the generation process. To bridge the gap between training capacity and inference efficiency, we employ a re-parameterization strategy to fold parameter matrices used during training into a compact static lookup table. By offloading the knowledge to ROM, MeKi decouples model capacity from computational cost, introducing zero inference latency overhead. Extensive experiments demonstrate that MeKi significantly outperforms dense LLM baselines with identical inference speed, validating the effectiveness of memory-based scaling paradigm for on-device LLMs. Project homepage is at https://github.com/ningding-o/MeKi.

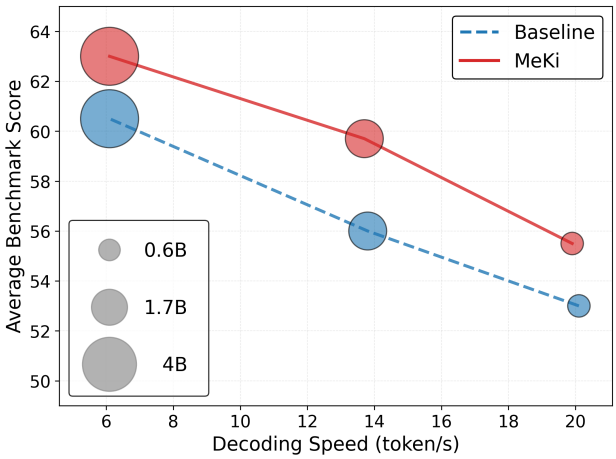

Figure 1: Memory-based scaling for on-device deployment. Scaling parameters of LLM makes inference much slower on NPU[1], which motivates our methodology of memory-based scaling by offloading static knowledge into Read-Only Memory (ROM). 1.7B-MeKi achieves an average zero-shot benchmark score of 59.7, statistically rivaling the 4B dense model's score of 60.5, while maintaining a 2.26× advantage in decoding speed.

[1] We measure the generation speed (token/s) on Qualcomm Snapdragon 8 Elite mobile platform with KV cache length being 10K.

1 Introduction

The scaling law (Kaplan et al., 2020; Hoffmann et al., 2022) for Large Language Models (LLMs) has become the de-facto methodology to achieve increasingly higher performance among all the flagship models. To follow this trajectory, some choose to crawl and synthesize massive training corpora (Raffel et al., 2020; Almazrouei et al., 2023; Penedo et al., 2024) for lengthened training stages. Alternatively, others choose to continuously increase the number of parameters within the Transformer architecture. This includes the previous SwiGLU (Shazeer, 2020) method which expands the feed-forward networks (FFNs), as well as the prevalently state-of-the-art Mixture-of-Experts (MoE) (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024) architecture which comprises multiple FFNs but only activates a sparse subset for each token. Recently, researchers find that scaling test-time FLOPs is also able to improve the performance. By exploring the reasoning path (Wei et al., 2022), verifying intermediate outputs (Cobbe et al., 2021), or employing search-based decoding strategies (Yao et al., 2023; Snell et al., 2024), models can effectively trade latency for accuracy. This approach shifts the focus from training-time weight updates to dynamic test-time computation during the deployment stage.

While these compute-heavy scaling strategies have achieved remarkable success in data-center environments, they encounter significant obstacles when deployed to edge devices. Enlarging the parameters of dense model increases the floating-point operations (FLOPs), leading to unacceptable latency and power consumption on mobile hardware, to which users are usually very sensitive. Figure 1 demonstrates the detailed inference speed of dense LLMs on the smartphone platform with on-device Qualcomm NPU (Qualcomm, 2025). Although MoE architectures reduce per-token FLOPs by sparsely activating experts, this architecture introduces substantial latency overhead due to the frequent loading of large and disjoint expert weights. The memory access pattern of MoE would becomes the primary latency bottleneck on resource-constrained hardware such as mobile phone, where ROM-RAM and RAM-NPU bandwidth is far more limited than server-grade GPUs in data centers.

Consequently, scaling LLMs for edge deployment requires a new paradigm shifted away from compute-centric design. Unlike matrix multiplications which are expensive in computation, we observed that memory lookups (*e.g.* reading from ROM storage) are relatively cheap and energy-efficient on modern mobile System-on-Chips (SoC). Besides, the ROM bandwidth is largely idle during model inference.This observation motivates a critical question:

> *Can we scale up the model capacity by using the storage space, without increasing the latency and FLOPs during inference?*

To answer this, we introduce **MeKi** (**M**emory-based **E**xpert **K**nowledge **I**njection), a novel LLM architecture which decouples the model capacity from computational cost. Just like MoE model retrieves specialized experts for different tokens, MeKi retrieves token-level dedicated knowledge vectors from a massive memory bank in every layer, injecting the learned knowledge into the hidden states.

To increase model capacity while maintaining efficiency, MeKi has different structures for training and inference. During the training stage, MeKi employs embedding-based memories along with non-linear projections to learn token-level expert knowledge representations. These features are fused with the hidden state via a low-rank gating mechanism to inject useful information into the hidden sequence. After training, we use re-parameterization technique to merge the complex projections into representations stored in the memory bank, which is offloaded to ROM. This turns the heavy online computation into a compact static lookup operation during inference. The MeKi system operates in parallel with the Transformer's FFN module, which allows for implicit layer expansion nearly without extra FLOPs overhead.

The contributions of this work are summarized as follows:

- We propose **MeKi**, a memory-centric scaling method, which decouples the LLM capacity growth from computation. We treat the layer-wise memory as a collection

of token-level experts that carry rich prior knowledge. We design a efficient fusion architecture parallel to the FFN module, using knowledge injection to expand model capacity.

- We introduce a re-parameterization strategy that balance the training-time capacity and test-time efficiency. Our strategy allows the use of complex non-linear projections during training to maximize feature representability, while these projections are then merged into a static embedding table for zero-cost inference.
- We validate the effectiveness of the proposed MeKi architecture across the scales of 0.6B, 1.7B, and 4B parameters. Our method outperforms the dense baseline across 10 widely-used benchmarks.
- We test the inference latency of MeKi architecture and its baseline counterpart on Qualcomm Snapdragon android hardware. Our method maintains the same inference speed with significant performance gains.

2 Related Works

Evolution of LLM Architectures. The trajectory of LLM development (Touvron et al., 2023; Achiam et al., 2023) has reached a critical turning point. While scaling laws continue to drive performance gains in massive data centers, the challenge of deploying these models on edge devices remains significantly underexplored. The Mixture-of-Experts (MoE) paradigm (Shazeer et al., 2017; Fedus et al., 2022; Liu et al., 2024) scales model capacity by sparsely activating parameter subsets. However, MoE introduces substantial inference overhead on edge hardware due to dynamic routing latency and memory fragmentation. In contrast, MeKi proposed in this paper eliminates online routing costs and enables seamless prefetching, making it more suitable for resource-constrained environments.

New Scaling Paradigm Beyond RAM. An upsurging research direction aims to scale model performance by leveraging external storage rather than increasing active memory (RAM) or FLOPs: *1) Retrieval-Augmented Performance Scaling* (Khandelwal et al., 2019; Borgeaud et al., 2022; Asai et al., 2024; Huang et al., 2024) that augments LLMs by retrieving information from massive corpora. *2) Memory-Centric Capacity Expansion*. Zeng et al. (2023) proposed LookupFFN which explored replacing the whole FFNs with lookup tables, and Sadhukhan et al. (2026) used lookup tables to replace part of the FFN. Ding et al. (2024) further replaced all linear layers in model with lookup tables. Per-Layer Embedding (DeepMind, 2025) (PLE) utilized tokenID-indexed embedding memory to expand the model depth, but is prone to forming information bottlenecks in the forward pass. The proposed MeKi advances beyond PLE by parallelizing embedding memory with FFN to serve as a capacity extender. Concurrent to our work, Engram (Cheng et al., 2026) utilizes N-gram statistics for phrase-level caching. Unlike Engram which relies on online hashing for knowledge retrieval, MeKi utilizes a low-rank gating mechanism to dynamically augment hidden states with token-level expert knowledge, providing superior contextual adaptation and lower latency for edge deployment.

3 Architecture

3.1 Overview

In this section, we detail the architecture of the proposed **M**emory-based **E**xpert **K**nowledge **I**njection (MeKi) system. We first provide a high-level overview of the framework, followed by a detailed formulation of each component. Section 3.2 details the composition of the Token-Level Expert during training stage. Section 3.3 details how to inject the retrieved knowledge into hidden state via a low-rank feature fusion mechanism. Section 3.4 proposes a re-parameterization strategy for inference optimization.

The core motivation of MeKi is to decouple the model capacity from computational cost by shifting the burden from floating-point matrix multiplications (FLOPs) to high-density memory lookups. Figure 2 illustrates the integration of MeKi within a standard Transformer

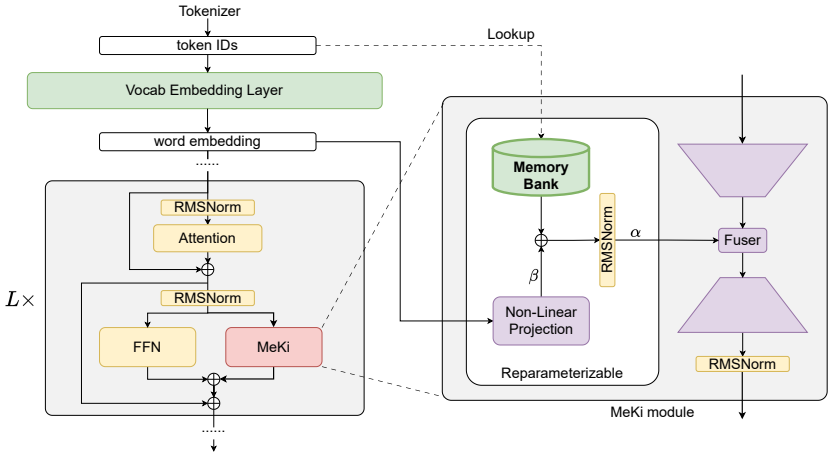

Figure 2: General overview of the proposed **MeKi** architecture. After training, all the re-parameterizable parts are merged into one static memory table to avoid FLOPs overhead.

layer. MeKi operates as a parallel branch to the standard Feed-Forward Network (FFN). For the l-th layer, MeKi retrieves a specific knowledge vector for every token from a layer-specific memory bank. This retrieval is then used to modulate the input hidden state to generate context-aware expert knowledge. The branch then projects the token knowledge back to the model dimension and adds it to the residual stream. Crucially, the architecture utilizes different parameter configurations for training and inference. During training, it employs complex non-linear projections to learn rich representations. Before deployment, a re-parameterization technique folds these complex computations into the memory bank weights, which guarantees that the inference latency in not compromised.

3.2 Memory-based Token-Level Expert

The MeKi module conceptualizes the embedding memory as a sparse token-level expert. Unlike Mixture-of-Experts (MoE) models that activate different network weights based on routing mechanism, MeKi retrieves expert knowledge directly from a Read-Only Memory (ROM) structure, ensuring that for any input token, only the specific vector corresponding to its ID is activated.

Given an input sequence with corresponding input token IDs $\mathbf{x} \in \{1, \ldots, |\mathcal{V}|\}^T$, T is the sequence length and $|\mathcal{V}|$ is the vocabulary size. For the l-th transformer layer, we define a layer-specific memory embedding matrix $\mathbf{M}^l \in \mathbb{R}^{|\mathcal{V}| \times d_{mem}}$, where d_{mem} is the memory dimension (*e.g.*, 128 or 256) and is typically much smaller than d_{model}. Given an input token ID x_t, the static retrieved memory vector $\mathbf{m}^l_{static}(x_t)$ is obtained via a standard embedding lookup:

$$\mathbf{m}^l_{static}(x_t) = \mathbf{M}^l[x_t]. \tag{1}$$

To further enhance the representation ability of the expert without increasing inference cost, we introduce a dynamic projection component during training. This component utilizes a non-linear projection function $\mathbf{G}^l(\cdot)$ (implemented as an SwiGLU in practice) acting on the global word embedding $E_{global} \in \mathbb{R}^{|\mathcal{V}| \times d_{model}}$. This allows the model to better synthesize layer-specific knowledge features from the shared semantic space:

$$\mathbf{m}^l_{dyn}(x_t) = \mathbf{G}^l(E_{global}[x_t]). \tag{2}$$

The final expert vector e^l_t for the t-th token at layer l is constructed by gathering the static prior knowledge and the dynamic projection. This fusion is stabilized by a normalization two learnable scalar α^l and β^l:

$$\mathbf{e}^l_t = \alpha^l \cdot \text{RMSNorm}\left(\mathbf{m}^l_{static}(x_t) + \beta^l \cdot \mathbf{m}^l_{dyn}(x_t)\right). \tag{3}$$

Here, β^l allows MeKi to learn a layer-wise combining ratio between the static learned knowledge and the dynamic projected feature, and α^l controls the layer-specific knowledge injection magnitude.

3.3 Knowledge Injection

Once the expert vector $\mathbf{e}_t^l$ is retrieved, it must be injected into the Transformer's hidden state flow. A critical design choice in MeKi is the fusion mechanism, which modulates how the retrieved expert knowledge $\mathbf{e}_t^l$ interacts with the t-th token hidden state $\mathbf{h}_t^l \in \mathbb{R}^{d_{model}}$. It's worth noting that, we let MeKi block share the same input hidden state $\mathbf{H}$ as the FFN block, where $\mathbf{H} = \{\mathbf{h}_1^l, \cdots, \mathbf{h}_T^l\}$ is the output of the preceding normalization layer.

We employ an additive gated fusion mechanism, which offers superior optimization stability compared to multiplicative gating. Specifically, we first generate a gate signal $\mathbf{g}_t$ by using the hidden state via a low-rank linear projection followed by a sigmoid activation:

$$\mathbf{g}_t^l = \sigma(\mathbf{W}_{gate}^l \mathbf{h}_t^l), \quad \mathbf{W}_{gate}^l \in \mathbb{R}^{d_{mem} \times d_{model}}. \tag{4}$$

The expert vector $\mathbf{e}_t^l$ is then modulated by adding this gate signal, implying that $\mathbf{g}_t^l$ serves as a contextualized channel-wise offset to adjust the expert knowledge:

$$\mathbf{v}_t^l = \mathbf{e}_t^l + \mathbf{g}_t^l. \tag{5}$$

The modulated vector $\mathbf{v}_t$ resides in the memory dimension d_{mem}. To inject it back into the main residual stream of the Transformer layer, we project it up to the hidden dimension d_{model} using another linear projection:

$$\mathbf{y}_t^l = \text{RMSNorm}\left(\mathbf{W}_{out}^l \mathbf{v}_t^l\right), \quad \mathbf{W}_{out}^l \in \mathbb{R}^{d_{model} \times d_{mem}}. \tag{6}$$

The final output of the MeKi module $\mathbf{y}_t^l$ is then added to the residual connection of the Transformer layer. If we denote the FFN block as $\mathbf{F}(\cdot)$ and the input sequence as $\mathbf{H}$, the output of the l-th Transformer layer is updated as:

$$\mathbf{H} = \text{RMSNorm}_{\text{pre\_ffn}}(\mathbf{A}) \tag{7}$$

$$\mathbf{H}' = \mathbf{F}(\mathbf{H}) + \mathbf{MeKi}(\mathbf{H}) + \mathbf{A}, \tag{8}$$

where $\mathbf{A}$ is the output sequence of the residual-add op after Attention module. This parallel structure allows MeKi to function as an implicit width expansion of the Transformer layer, enhancing capacity without affecting the original connection path of the FFN module.

3.4 Re-Parameterization for Faster Inference

While the non-linear projection $\mathbf{G}^l(\cdot)$ in the MeKi module significantly boosts the model's learning capacity, it also introduces additional FLOPs during the forward pass. To achieve the goal of zero-cost inference, we propose a re-parameterization technique that merges the dynamic projection $\mathbf{m}_{dyn}^l(\cdot)$ into the embedding table $\mathbf{M}^l$ of static memory.

Observing that the dynamic projection $\mathbf{G}^l(\cdot)$ operates solely on the global word embedding weights E_{global}, which are static constants after training, we can pre-compute the output of this projection. Specifically, we define a new embedding table $\tilde{\mathbf{M}}^l$ as follows:

$$\tilde{\mathbf{M}}^l = \text{Re-parameterize}\left(\mathbf{M}^l, E_{global}, \mathbf{G}^l\right) \tag{9}$$

$$= \alpha^l \cdot \text{RMSNorm}\left(\mathbf{M}^l + \beta^l \cdot \mathbf{G}^l(E_{global})\right). \tag{10}$$

By replacing the original memory weight $\mathbf{M}^l$ with $\tilde{\mathbf{M}}^l$, the derivation of token-level expert vector at inference time is simplified drastically. The term $\mathbf{G}^l(E_{global}[x_t])$ gets absorbed

into the new lookup table, eliminating the need for any matrix multiplication during the generation of expert knowledge vector $\mathbf{e}_t^l$.

Consequently, the knowledge retrieval at inference-time becomes:

$$\tilde{\mathbf{e}}_t^l = \tilde{\mathbf{M}}^l[x_t], \quad \tilde{\mathbf{M}}^l \in \mathbb{R}^{|\mathcal{V}| \times d_{mem}}, \tag{11}$$

followed by the lightweight fusion step. This fusion step involves only two small matrix multiplications of size $d_{mem} \times d_{model}$ and an addition. Therefore, the inference process of MeKi is:

$$\mathbf{g}_t^l = \sigma(\mathbf{W}_{gate}^l \mathbf{h}_t^l), \tag{12}$$

$$\mathbf{v}_t^l = \tilde{\mathbf{e}}_t^l + \mathbf{g}_t^l, \tag{13}$$

$$\mathbf{y}_t^l = \text{RMSNorm}\left(\mathbf{W}_{out}^l \mathbf{v}_t^l\right) \tag{14}$$

3.5 Computational Complexity Analysis

We demonstrate the efficiency of the proposed MeKi by analyzing the FLOPs statistic per layer per token.

Training FLOPs of MeKi. The non-linear projection $\mathbf{G}^l$ which generates dynamic knowledge feature $\mathbf{m}_{dyn}^l(x_t)$ is implemented as an SwiGLU with intermediate size $\frac{d_{model}}{2}$. The FLOPs of $\mathbf{G}^l$ per token are $O(d_{model}^2 + \frac{d_{model}}{2} \cdot d_{mem})$. After considering the two low-rank gating projections $\mathbf{W}_{gate}^l$ and $\mathbf{W}_{out}^l$, the per-token FLOPs of MeKi module are $O(d_{model}^2 + \frac{5}{2} d_{model} \cdot d_{mem})$ during training stage.

Re-Parameterized Inference FLOPs. After the heavy projection $\mathbf{G}^l$ is removed, the remaining operations are the embedding lookup (negligible I/O cost) and the low-rank gating projections with FLOPs of $O(d_{model} \cdot d_{mem})$ level. Since the memory dimension $d_{mem} \ll d_{model}$ (*e.g.* 128 vs 2048), the re-parameterized inference effectively shifts the complexity from matrix multiplication to memory access bandwidth.

On-device Circumstance. For a 28-layer model with $d_{mem} = 256$, the memory weights to be moved from ROM is only $d_{mem} \times L = 7168$, which corresponds to 14KB per token in float16 format. On modern mobile SoC with NPU, embedding tables are typically cached in high-speed memory where ROM bandwidth is abundant (*e.g.* UFS-4.0 delivers 4.2GB/s reading speed (Semiconductor, 2025)). Therefore, the model obtained after re-parameterization has almost the same token generation latency compared to the baseline model without MeKi.

4 Experiments

In this section, we evaluate the proposed architecture to demonstrate that MeKi effectively decouples parametric memory from computational cost, achieving superior performance on comprehensive benchmarks without incurring inference latency overhead.

4.1 Experimental Setup

Datasets and Baselines. We conduct pre-training on the *FineWeb-Edu-Dedup* dataset (Ben Allal et al., 2024), which comprises high-quality educational contents collected from internet. We randomly sampled 50 billion tokens from this dataset and use the same 50B-subset to pre-train all the models in this paper for fair comparisons. The models are implemented using the Megatron-LM framework (Shoeybi et al., 2019). We adopt the architecture of Qwen3-0.6B, -1.7B, and -4B (Team, 2025) as our primary dense baselines.

Table 1: **Zero-shot performance on downstream tasks.** We compare MeKi against the dense baseline by *training from scratch* for 50B tokens. MeKi achieves the best average performance across all the model scales, demonstrating the efficacy of our proposed architecture.

Models	# ROM Weights	Speed (token/s)	ARC-E	ARC-C	BoolQ	COPA	HellaSwag	LAMBADA	OBQA	PIQA	SCIQ	WinoGrande	Avg.
0.6B In-RAM Parameters													
Baseline	-	20.1	30.5	56.0	58.3	71.0	45.7	35.5	**34.8**	68.7	77.3	52.6	53.0
MeKi	0.5B	19.9	**33.6**	**60.2**	**63.0**	**72.0**	**49.2**	**39.8**	34.6	**70.2**	**78.4**	**53.8**	**55.5**
1.7B In-RAM Parameters													
Baseline	-	13.8	34.4	61.7	58.9	69.0	51.7	41.3	37.6	70.8	80.6	53.6	56.0
MeKi	1.1B	13.7	**37.9**	**66.2**	**62.4**	**74.0**	**56.6**	**45.6**	**39.0**	**71.7**	**85.4**	**58.7**	**59.7**
4B In-RAM Parameters													
Baseline	-	6.1	38.0	66.3	62.7	**80.0**	57.9	45.6	39.4	72.9	84.4	58.1	60.5
MeKi	2.8B	6.1	**42.2**	**70.2**	**64.4**	77.0	**62.3**	**50.1**	**40.2**	**75.4**	**87.2**	**61.6**	**63.0**

Training Details. All models are trained from scratch with the AdamW optimizer ($\beta_1 = 0.9, \beta_2 = 0.95$), using BFloat16 mixed-precision. To ensure training stability, we apply a weight decay of 0.1 and employ gradient clipping with a global norm threshold of 1.0. We use a cosine learning rate schedule with a warm-up phase of 500 steps. The peak and minimum learning rate is set to 4.0×10^{-4} and 2.0×10^{-4}, respectively. We use the global batch size of 256 and the sequence length of 4,096. Each training step contains 1M tokens. All the models are trained for 50,000 steps. Detailed architecture and hyperparameter configurations are summarized in Table 5.

Evaluation Benchmarks. To evaluate our model's performance, we conduct evaluations across ten recognized benchmarks, categorized by required reasoning capabilities following LLaMA (Touvron et al., 2023): *1) Science & Knowledge*: ARC-E/C (Clark et al., 2018), OBQA (Mihaylov et al., 2018), and SciQ (Welbl et al., 2017) are used to test factual retrieval and scientific reasoning; *2) Commonsense & Causality*: PIQA (Bisk et al., 2020), COPA (Roemmele et al., 2011), and HellaSwag (Zellers et al., 2019) assess physical commonsense, causal relationships, and plausible scene continuation; *3) Comprehension & Logic*: BoolQ (Clark et al., 2019) and WinoGrande (Sakaguchi et al., 2021) evaluate boolean reading comprehension and robust pronoun resolution; *4) Language Modeling*: LAMBADA (Paperno et al., 2016) measures the model's proficiency in predicting words based on broad discourse context.

4.2 Experimental Result

We evaluate the zero-shot performance / accuracy for pre-trained checkpoints on the above benchmarks.

Comparison to Dense Baselines. Table 1 presents a comprehensive comparison between MeKi and dense baselines across diverse benchmarks. The results explicitly validate that separating memory capacity from computational logic yields significant gains without increasing FLOPs.

- *Knowledge-Intensive Tasks:* MeKi demonstrates its strongest advantage in tasks requiring factual recall. On ARC-Challenge, MeKi-1.7B achieves a score of 37.9, surpassing the 1.7B baseline (34.4) by +3.5 points and effectively matching the performance of the significantly larger 4B dense model (38.0). Similarly, on SciQ, MeKi-1.7B reaches 85.4, outperforming the baseline by 4.8 points. This suggests that the ROM effectively functions as an extended key-value store for static world knowledge, relieving the FFN parameters from the burden of memorization.
- *Reasoning and Context:* In reasoning-heavy tasks like BoolQ and HellaSwag, MeKi-1.7B scores 62.4 and 56.6 respectively, consistently outperforming the baseline (58.9 and 51.7). Notably, on the LAMBADA language modeling benchmark, MeKi-1.7B achieves a score of 45.6, which is identical to the 4B baseline model (45.6). This indicates that the injected "expert vectors" provide crucial semantic anchoring for long-range dependency prediction, effectively simulating the capacity of a 4B parameter model using only 1.7B active parameters and external memory.

Table 2: **Zero-shot performance on downstream tasks.** We compare MeKi against PLE (DeepMind, 2025) and Engram (Cheng et al., 2026) with similar memory size offloaded to ROM. * We re-produce these results, see footnotes on this page for details.

Benchmark	0.6B Scale			1.7B Scale		
	PLE*	Engram*	MeKi	PLE*	Engram*	MeKi
#RAM Params	0.61B	0.60B	0.60B	1.76B	1.75B	1.75B
#ROM Weights	0.54B	0.62B	0.54B	1.09B	1.24B	1.09B
ARC-C	31.5	31.4	**33.6**	33.7	36.8	**37.9**
ARC-E	58.8	**60.3**	60.2	62.3	65.8	**66.2**
BoolQ	57.9	61.9	**63.0**	61.6	60.9	**62.4**
COPA	67.0	68.0	**72.0**	73.0	72.0	**74.0**
HellaSwag	46.6	45.3	**49.2**	52.7	54.0	**56.6**
LAMBADA	36.1	34.5	**39.8**	41.5	42.3	**45.6**
OBQA	**35.0**	33.6	34.6	36.4	36.0	**39.0**
PIQA	69.1	69.2	**70.2**	69.7	**71.9**	71.7
SCIQ	**80.3**	78.5	78.4	81.8	83.0	**85.4**
WinoGrande	52.9	**54.5**	53.8	57.4	56.3	**58.7**
Average	53.5	53.7	**55.5**	57.0	57.9	**59.7**

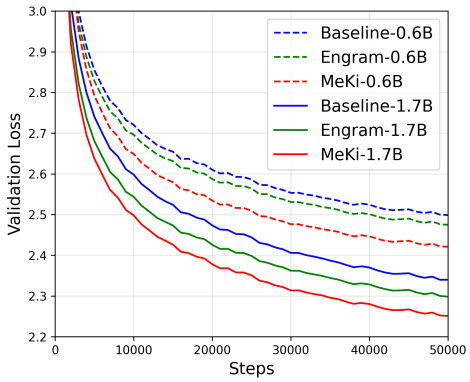

Figure 3: The comparison of validation loss for different methods where MeKi outperforms both baseline and Engram.

Comparison to Sparse Memory Architectures. To evaluate the efficacy of our proposed architecture, we compare[2] MeKi with PLE (DeepMind, 2025) and Engram (Cheng et al., 2026). As shown in Table 2, MeKi-1.7B attains an average score of 59.7, outperforming PLE and Engram by 2.7 and 1.8 points, respectively. Figure 3 illustrates the validation loss curves, where MeKi outperforms other approaches by a large margin for both the 0.6B and 1.7B scale. These results demonstrate that MeKi is not merely a storage extension, but a more effective mechanism for integrating large-scale static knowledge into the transformer backbone.

Inference Latency. In Table 1, we report the generation speed (token/s) of our MeKi architecture and its dense baseline counterpart, we use android smartphone with Qualcomm Snapdragon 8 Elite platform and set the KV cache length to be 10K. During inference, MeKi maintains the same number of active parameters in RAM as the dense baseline. By offloading the memory weights to ROM storage space after re-parameterization, and using asynchronous prefetching directly via token ID, we achieve nearly zero latency overhead.

[2] We implement PLE based on the official code of Gemma-3n and the Engram method based on the official github repository.

Table 3: **Zero-shot performance with different types of memory.** The two variants are also based on 0.6B baseline model.

Model	$\mathbf{m}^l_{static}$	$\mathbf{m}^l_{dyn}$	ARC-E	ARC-C	BoolQ	COPA	HellaSwag	LAMBADA	OBQA	PIQA	SCIQ	WinoGrande	Avg.
Baseline-0.6B	×	×	30.5	56.0	58.3	71.0	45.7	35.5	34.8	68.7	77.3	52.6	53.0
Static-Only-0.6B	✓	×	32.8	59.7	61.4	71.0	49.0	37.4	**36.0**	69.1	79.2	52.4	54.8
Dynamic-Only-0.6B	×	✓	31.2	57.5	62.0	71.0	47.7	39.0	35.6	69.3	**79.5**	**54.1**	54.7
MeKi-0.6B	✓	✓	**33.6**	**60.2**	**63.0**	**72.0**	**49.2**	**39.8**	34.6	**70.2**	78.4	53.8	**55.5**

Table 4: **Zero-shot performance with MeKi module incorporated at different positions.** All models are based on MeKi-0.6B.

Position	ARC-E	ARC-C	BoolQ	COPA	HellaSwag	LAMBADA	OBQA	PIQA	SCIQ	WinoGrande	Avg.
① Parallel to FFN	**33.6**	60.2	**63.0**	**72.0**	**49.2**	**39.8**	34.6	**70.2**	78.4	53.8	**55.5**
② Parallel to Attn	31.4	**61.0**	61.7	71.0	48.4	38.6	**36.0**	69.0	78.4	**55.8**	55.1
③ After Attn	31.7	60.2	58.9	71.0	48.6	37.4	35.8	69.0	**81.7**	54.9	54.9
④ After FFN	33.3	60.4	61.4	71.0	46.9	38.7	33.2	69.5	79.4	53.3	54.7

5 Ablation Study and Analysis

In this section, we analyze the critical architectural choices of MeKi to understand their contribution to the overall performance. All the experiments conducted in this section follow the same setting in Section 4.1.

5.1 Effectiveness of Static and Dynamic Memory

To verify the individual contribution of the two distinct memory components within MeKi module, we selectively disable the static memory retrieval $\mathbf{m}^l_{static}$ or the dynamic memory projection $\mathbf{m}^l_{dyn}$ during training. We summarize the results in Table 3 and provides insights below.

Static-Only. Firstly, we evaluate the *Static-Only* variant, which solely relies on the trainable memory embedding table $\mathbf{M}^l$. Therefore Equation (3) reduces to

$$\mathbf{e}^l_t = \alpha^l \cdot \text{RMSNorm}\left(\mathbf{M}^l[x_t]\right). \tag{15}$$

Compared to the 0.6B baseline, this variant achieves significant improvement (average score raised from 53.0 to 54.8). This result indicates the static memory effectively learns the token-level priors and factual knowledge directly in the embedding space.

Dynamic-Only. Secondly, we examine the *Dynamic-Only* variant, where the static memory table $\mathbf{M}^l$ is deleted. The token-level expert is derived by applying a non-linear projection $\mathbf{G}^l$ to the global word embeddings. In this situation, Equation (3) reduces to

$$\mathbf{e}^l_t = \alpha^l \cdot \text{RMSNorm}\left(\beta^l \cdot \mathbf{G}^l(E_{global}[.x_t])\right). \tag{16}$$

This variant performs similarly to the Static-Only variant. This suggests that complex non-linear transformation is capable of synthesizing expressive layer-specific features from the global semantic space, providing the model with enhanced representability without layer-specific memory.

MeKi-0.6B integrates both memory components via the learnable coefficients α^l and β^l. It surpasses both Static-Only and Dynamic-Only variants across the majority of benchmarks. The consistent gain (+0.7 over Static-Only and +0.8 over Dynamic-Only) suggests that the static memory and dynamic projection capture complementary information. Their combination allows the model to maximize the utilization of the storage budget for knowledge injection.

5.2 Optimal Position for Memory Injection

We investigate the optimal position where the MeKi module should be incorporated within a Transformer layer. As illustrated in Section 5.2, we compare 4 different placement, ① Parallel to FFN, ② Parallel to Attention, ③ After Attention, ④ After FFN.

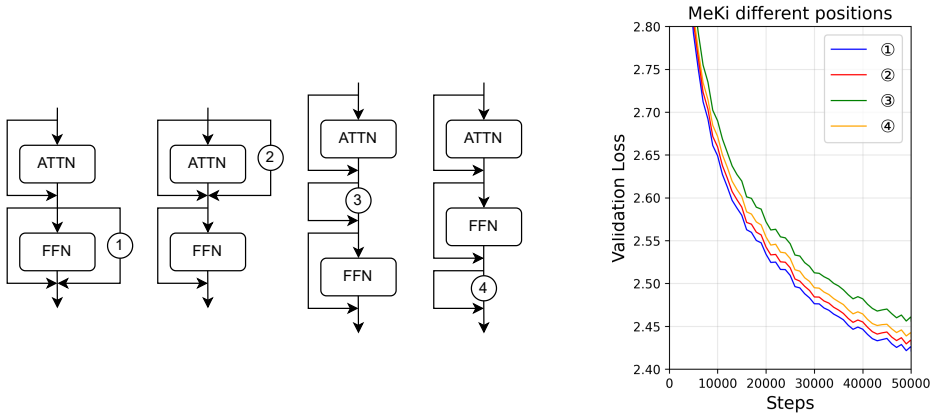

Figure 4: (Left) We choose four potential positions to incorporate the MeKi module within our architecture. (Right) Validation loss for different position settings.

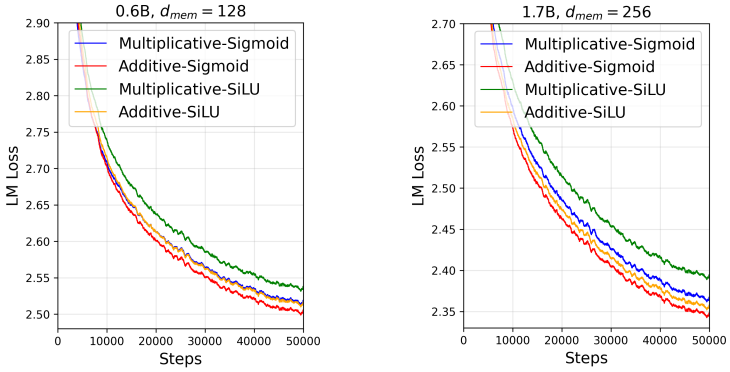

Figure 5: Validation loss over different feature fusion strategies.

Table 4 demonstrates that the option ①, Parallel to FFN, achieves the best scores averaged from 10 downstream tasks. Section 5.2 also shows that option ① has the best validation loss during training. For the option ③ and ④ that achieve the worst performance, we conjecture that the low-rank gating structure inside the MeKi leads to information bottleneck when used alone, causes performance drop. The option ② is sub-optimal because the Attention is responsible for building the global dependencies among different tokens in a sequence. The token-level expert knowledge at position ② would play a much weaker role than the situation where it serves as an capacity augmentation parallel to FFN.

5.3 Optimal Feature Fusion

The integration of the retrieved knowledge vector $\mathbf{e}_t^l$ into the transformer block is critical to maximize the memory expert's utilization. MeKi operate in the low-rank space in order to save FLOPs while injecting expert knowledge $\mathbf{e}_t^l$ into the contextualized token hidden state $\mathbf{h}_t^l$:

$$\mathbf{v}_t^l = \mathbf{e}_t^l + \mathbf{g}_t^l = \mathbf{e}_t^l + \sigma(\mathbf{W}_{gate}^l \mathbf{h}_t^l). \tag{17}$$

In total, we explored four different feature fusion strategies as follows. We omit subscript and superscript for simplicity.

- Additive-Sigmoid: $\mathbf{v} = \mathbf{e} + \sigma(\mathbf{W}_{gate}\mathbf{h})$,
- Multiplicative-Sigmoid: $\mathbf{v} = \mathbf{e} \odot \sigma(\mathbf{W}_{gate}\mathbf{h})$,
- Additive-SiLU: $\mathbf{v} = \mathbf{e} + \text{SiLU}(\mathbf{W}_{gate}\mathbf{h})$,
- Multiplicative-SiLU: $\mathbf{v} = \mathbf{e} \odot \text{SiLU}(\mathbf{W}_{gate}\mathbf{h})$.

We train all these variants from scratch based on MeKi-0.6B and MeKi-1.7B, and show the corresponding training loss curves in Figure 5. As demonstrated in the loss curves, the "Additive-Sigmoid" method achieves the best performance among these fusion strategies.

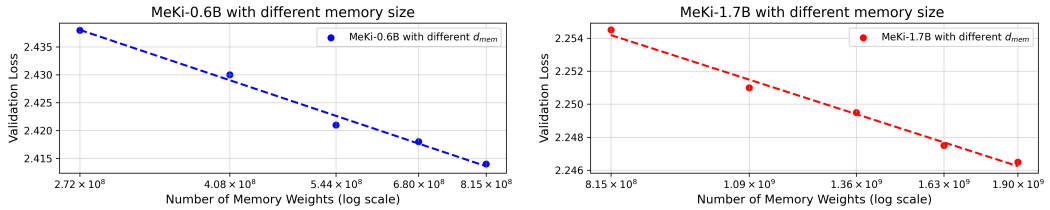

Figure 6: The final validation loss values with different memory sizes. All the models are trained from scratch for 50 billion tokens.

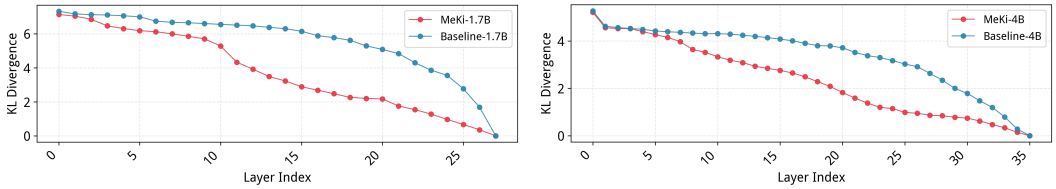

Figure 7: Layer-wise KL Divergence Analysis. MeKi exhibits a lower divergence across all layers, demonstrating its ability to accelerate prediction convergence.

5.4 Scaling Law of Memory Size

We investigate the scaling properties of the proposed MeKi architecture by explore the impact of the memory dimension d_{mem} used by the knowledge vectors. For MeKi-0.6B, we vary the d_{mem} in [64, 96, 128, 160, 192], which directly leads to the memory size of MeKi-0.6B being [2.72×10^8, 4.08×10^8, 5.44×10^8, 6.80×10^8, 8.15×10^8] respectively. We compute the number of Memory Weights according to $L \times |\mathcal{V}| \times d_{mem}$, where L is the number of transformer layers. Similarly for MeKi-1.7B, we vary the d_{mem} in [192, 256, 320, 384, 448], which leads to the memory size of MeKi-1.7B being [8.15×10^8, 1.09×10^9, 1.36×10^9, 1.63×10^9, 1.90×10^9] respectively.

We illustrate the final validation loss of different models under the same training data budget and the fixed active parameter budget in Figure 6. As anticipated, we observe that the model performance consistently exhibits a **log-linear** trend with respect to the memory size. In order to balance the storage cost (*e.g.* ROM space) and model capacity, we select the dimension $d_{mem} = 128$ for MeKi-0.6B and $d_{mem} = 256$ for MeKi-1.7B as the optimal trade-off for our main experiments.

5.5 Accelerating Prediction Convergence

We employ the LogitLens (nostalgebraist, 2020) to examine how predictions evolve across the model's layers. We input the intermediate hidden states of each layer into the final classifier head, we calculate the KL divergence between these early predicted and the final output distribution. This measures convergence of latent representations toward their final predicted state.

Figure 7 reports the statistics by averaging the results of 10K tokens sampled from validation set. Compared to the baseline, MeKi exhibits a systematically lower KL divergence across all layers. The consistently lower values indicate that MeKi facilitates a more rapid alignment of intermediate feature distributions with the final output. This observation suggests that by retrieving knowledge vector from the layer-specific memory bank, MeKi accelerates prediction convergence, enabling the model to reach high-confidence states earlier inside the network architecture.

5.6 Scaling Training-Time FLOPs

During training, the projector $\mathbf{G}^l(\cdot)$ dynamically maps the global word embeddings to the MeKi's layerwise lower-dimensional knowledge space. We try to use a simple linear

projection layer as $\mathbf{G}^l$ to replace the current SwiGLU function. We observe degradation in both the training loss curve and the downstream performance across all model sizes. During the inference stage, no matter how the projection $\mathbf{G}^l$ is modeled, it will be absorbed into the final embedding memory $\tilde{\mathbf{M}}^l$ by reparameterization techniques, leading to no computation overhead at all. This suggests that increasing the FLOPs of the training phase is still helpful to the final performance even though these FLOPs are never used for inference.

6 Conclusion

MeKi introduces a new memory-based paradigm for scaling LLM, decoupling model capacity from computational cost by leveraging abundant ROM space to circumvent the limitations of on-device resources constraints. By re-parameterizing the computational overhead of training into static memory lookup tables, MEKI achieves zero inference latency overhead while effectively expanding on-device model capacity via storage. Extensive experiments demonstrate that our approach enables a 1.7B model to rival the performance of a 4B dense model while maintaining identical decoding speed on Qualcomm Snapdragon hardware. Our findings validate that shifting the scaling bottleneck from computation to memory is a highly effective strategy for powerful on-device AI.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. 2024.

Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Smollm-corpus, 2024. URL https://huggingface.co/datasets/HuggingFaceTB/smollm-corpus.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.

Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Google DeepMind. Gemma 3n. 2025. URL https://ai.google.dev/gemma/docs/gemma-3n.

Ning Ding, Yehui Tang, Haochen Qin, Zhenli Zhou, Chao Xu, Lin Li, Kai Han, Heng Liao, and Yunhe Wang. Memoryformer: Minimize transformer computation by removing fully-connected layers. *Advances in Neural Information Processing Systems*, 37:20259–20275, 2024.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Zihao Huang, Qiyang Min, Hongzhi Huang, Defa Zhu, Yutao Zeng, Ran Guo, and Xun Zhou. Ultra-sparse memory network. *arXiv preprint arXiv:2411.12364*, 2024.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*, 2019.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.

nostalgebraist. Interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 1525–1534, 2016.

Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

Qualcomm. Qualcomm snapdragon mobile processors. 2025. URL https://www.qualcomm.com/processors/mobile-processors/.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*, pp. 90–95, 2011.

Ranajoy Sadhukhan, Sheng Cao, Harry Dong, Changsheng Zhao, Attiano Purpura-Pontoniere, Yuandong Tian, Zechun Liu, and Beidi Chen. Stem: Scaling transformers with embedding modules. *arXiv preprint arXiv:2601.10639*, 2026.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

Samsung Semiconductor. Mobile storage's big leap forward. 2025. URL https://semiconductor.samsung.com/estorage/ufs/ufs-4-0/.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. *URL https://arxiv.org/abs/2408.03314*, 20, 2024.

Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Johannes Welbl, Nelson F Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*, 2017.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. *URL https://arxiv. org/abs/2305.10601*, 3:1, 2023.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

Zhanpeng Zeng, Michael Davies, Pranav Pulijala, Karthikeyan Sankaralingam, and Vikas Singh. Lookupffn: making transformers compute-lite for cpu inference. In *International Conference on Machine Learning*, pp. 40707–40718. PMLR, 2023.

A Appendices

A.1 Detailed Model Architecture and Hyper Parameters

Table 5: Detailed model architecture information and training hyper parameters.

Hyper-Parameters	**MeKi-0.6B**	**MeKi-1.7B**	**MeKi-4B**
# RAM Params	0.60B	1.75B	4.12B
# ROM Weights	0.54B	1.09B	2.80B
# Training Tokens	50B	50B	50B
Layers	28	28	36
Hidden Size d_{model}	1024	2048	2560
MeKi Dim d_{mem}	128	256	512
Intermediate Size	3072	6144	9728
Attention module	Group Query Attention		
RoPE θ	500000		
Training Steps θ	50000		
Sequence Length	4096		
Vocab Size	151680		
Batch Size	256		
Base Learning Rate	4e-4		
Lr Scheduler	Cosine		
Adam β	(0.9, 0.95)		
Weight Decay	0.1		
Gradient Clip	1.0		

A.2 Detailed Statistics for NPU Inference

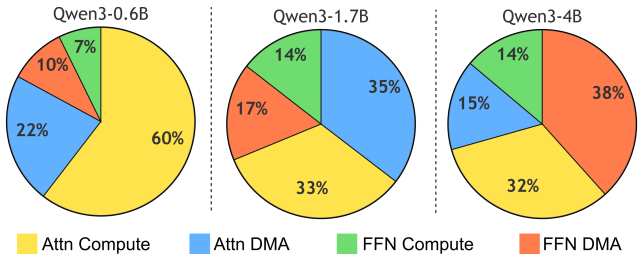

Figure 8: The percentage of time consumption of different model components during NPU inference.

For small-sized Qwen3 model family, we measure the percentage of time consumption for different model components during the on-device inference process. We get the statistics by measuring the generation speed (token/s) on Qualcomm Snapdragon 8 Elite with KV cache length being 10K.

The ***DMA*** part refers to the time spent on moving the corresponding parameters from RAM to NPU. During inference, the Qwen3-4B model spends 38% of its time moving FFN parameters from RAM to NPU. Besides, time spent on FFN computation is actually less than the parameter movement, where 32% of the inference time is used for FFN computation. The DMA time becomes a major bottleneck why scaling the number of parameters of dense LLM makes inference extremely slower on NPU. Deploying MoE architecture requires much more DMA time than dense LLM.